



МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет Информационных технологий

Кафедра Информатики и информационных технологий

направление подготовки

09.03.02 «Информационные системы и технологии»

ОТЧЕТ

по проектной практике

Студент: Козлитина Полина Романовна **Группа:** 251–332

Место прохождения практики: Московский Политех, кафедра
«Информатика и информационные технологии»

Отчет принят с оценкой _____ **Дата** _____

Руководитель практики: Семенова В. В.

Оглавление

<u>Введение</u>	<u>3</u>
<u>1. Общая информация о проекте</u>	<u>4</u>
<u>2. Общая характеристика деятельности организации</u>	<u>5</u>
<u>3. Описания задания по проектной практике</u>	<u>7</u>
<u>4. Описание достигнутых результатов по проектной практике</u>	<u>8</u>
<u>Заключение</u>	<u>10</u>
<u>Список использованной литературы</u>	<u>11</u>
<u>ПРИЛОЖЕНИЯ</u>	<u>12</u>
<u>ПРИЛОЖЕНИЕ 1. Исходный код программного модуля «PoLLelya»</u>	<u>12</u>
<u>ПРИЛОЖЕНИЕ 2. Демонстрация работы интерфейса и функций редактора.</u>	<u>20</u>

Введение

Сегодня трудно представить учёбу или работу без создания текстовых документов. Мы постоянно делаем отчёты, оформляем листинг кода, записываем свои идеи. Несмотря на присутствие монополиста MS Word, некоторые пользователи отдают предпочтение минималистичному интерфейсу Блокнота или Simplenote.

Язык программирования Python является одним из самых популярных средств разработки программного обеспечения благодаря простоте синтаксиса, широкому набору библиотек и возможности создания графических пользовательских интерфейсов. Использование Python позволяет разрабатывать приложения различного уровня сложности.

Актуальность данной работы заключается в необходимости изучения принципов создания прикладных программ с графическим интерфейсом, а также в приобретении практических навыков разработки программного обеспечения. Создание текстового редактора позволяет рассмотреть основные этапы проектирования программы: разработку интерфейса, реализацию функциональных возможностей, обработку пользовательских действий и организацию работы с текстовыми файлами.

1. Общая информация о проекте

Название проекта: «PoLLelya».

Цель проекта: разработка кроссплатформенного текстового редактора с адаптивным графическим интерфейсом и расширенными возможностями динамического форматирования текста.

Задачи проекта:

1. Анализ инструментов: Изучение возможностей библиотеки tkinter и механизмов низкоуровневого управления текстовым контентом через виджет ScrolledText.
2. Проектирование UI/UX: Разработка уникального пользовательского интерфейса, ориентированного на темную цветовую схему (Dark Mode), с использованием кастомных навигационных панелей (сайдбара).
3. Разработка системы форматирования: Реализация алгоритма комбинированного тегирования для управления начертанием (Bold, Italic), подчеркиванием и параметрами шрифта (размер, гарнитура) без потери свойств при их наложении.
4. Событийное управление: Настройка системы горячих клавиш и перехвата событий ввода для обеспечения непрерывности процесса редактирования.
5. Реализация логики управления данными: Создание отказоустойчивых механизмов открытия, сохранения и создания файлов, а также системы предотвращения потери данных при закрытии приложения.
6. Оптимизация состояний: Реализация динамического переключения тем оформления (Dark/Light Mode) с сохранением метаданных форматирования документа в оперативной памяти.

2. Общая характеристика деятельности организации

Наименование заказчика: Московский политехнический университет

Организационная структура Московского политехнического университета (Московского Политеха) включает несколько ключевых уровней:

- органы управления: ректор, ректорат, наблюдательный и учебный советы;
- факультеты и институты: Факультет информационных технологий, Факультет машиностроения, Факультет экономики и управления и др.;
- кафедры: кафедра «Информатики и информационных технологий» и др.;
- лаборатории и другие подразделения;
- филиалы: Чебоксарский институт, Рязанский институт, Коломенский институт и др.

Московский Политех осуществляет подготовку специалистов по широкому спектру направлений, включая информационные технологии, машиностроение, экономику, дизайн, транспорт и другие. Университет реализует образовательные программы бакалавриата, магистратуры и аспирантуры, а также ведёт активную научную и инновационную деятельность.

Кафедра «Информатика и информационные технологии» занимается:

- подготовкой студентов по IT-специальностям;
- разработкой и внедрением новых образовательных программ в области информатики;
- проведением научных исследований в сфере информационных технологий;
- организацией проектной и практической деятельности студентов;

- сотрудничество с предприятиями и организациями по вопросам цифровизации и автоматизации процессов.

3. Описания задания по проектной практике

В рамках прохождения проектной практики перед нами была поставлена задача разработать программу, предназначенную для создания, редактирования и сохранения текстовых документов.

Задание предполагает создание самостоятельного приложения с графическим интерфейсом, реализующего базовый набор функций для работы с текстом. Ключевым требованием является реализация следующих сценариев использования:

1. Создание и редактирование: обеспечение возможности ввода текста с клавиатуры, его удаления, копирования, вставки и перемещения.
2. Работа с файлами: реализация механизмов для сохранения текущего документа в файл на диске и открытия существующего файла для дальнейшего редактирования.
3. Формирование интерфейса: разработка интуитивно понятного графического интерфейса, включающего основное рабочее поле для текста и элементы управления (меню, кнопки).

В качестве основного инструмента разработки является язык Python и соответствующая библиотека для создания графического интерфейса. Задание не ограничивается только написанием программного кода, но также включает в себя этап отладки и тестирования работоспособности реализованных функций.

4. Описание достигнутых результатов по проектной практике

В процессе реализации проекта работа была разделена на три этапа: проектирование интерфейса, разработка логики обработки текста и финальная отладка. Для организации процесса разработки был использован Git-репозиторий. Это позволило структурировать этапы написания кода, отслеживать историю изменений и обеспечить сохранность промежуточных результатов работы.

На первом этапе был создан макет «PoLLelya». Вместо стандартных меню был использован боковой сайдбар и «парящий» тулбар. Основной сложностью здесь стала верстка элементов в Tkinter так, чтобы они выглядели современно. Для создания эффекта «листа бумаги» в темном пространстве были подобраны специфические значения внутренних отступов `padx` и `pady` для виджета `ScrolledText`. Также была определена цветовая палитра приложения (Листинг 1).

Листинг 1. Конфигурация цветовой схемы интерфейса

```
self.colors = {  
    "bg": "#1a1a1e",  
    "surface": "#252529",  
    "accent": "#7c4dff",  
    "text": "#e1e1e1",  
    "gray": "#626266",  
    "border": "#333338"  
}
```

Второй этап — разработка системы форматирования — выявил техническую проблему: стандартный виджет текста в Python не сохраняет активное состояние шрифта после изменения позиции курсора. Проблема была решена через перехват события нажатия клавиш и принудительное применение накопленного стека стилей к вставляемому символу, что исключило сброс форматирования при перемещении курсора.

На этапе реализации функций выравнивания мы столкнулись с ограничением библиотеки: значение `justify` в Tkinter поддерживает только три режима: `left`, `center`, `right`. Попытки внедрения «по ширине» привели к нестабильному отображению символов и нарушению логики работы индексов текста, в связи с чем было принято решение ограничиться тремя стабильно работающими режимами, гарантирующими корректное отображение документа.

Особое внимание было уделено иерархии тегов. Поскольку в Tkinter теги, изменяющие параметры шрифта, конфликтуют между собой, была реализована система динамической генерации имен тегов. Это позволило сохранять начертание шрифта при изменении его размера или гарнитуры.

В ходе отладки была решена проблема потери форматирования при динамической смене темы оформления. Для этого был использован метод `.dump()`, позволяющий извлечь текст вместе с метаданными тегов, и написан алгоритм регенерации конфигураций тегов при пересборке интерфейса.

Визуальные результаты работы приложения и примеры форматирования текста представлены в Приложении 2. Полный исходный код программы приведен в Приложении 1.

Заключение

Результатом практики стал работоспособный программный продукт – текстовый редактор «PoLLelya», полностью соответствующий исходному заданию и обладающий расширенным функционалом форматирования.

В ходе работы была успешно реализована многоуровневая система тегирования, позволившая преодолеть стандартные ограничения библиотеки Tkinter в области наследования шрифтовых стилей. Главная ценность проекта заключается в нахождении баланса между минималистичным дизайном и технической сложностью: удалось доказать, что использование кастомных систем обработки событий позволяет создавать на базе консервативных библиотек современные и отзывчивые интерфейсы.

Разработанный механизм динамической смены тем с сохранением метаданных форматирования подтвердил надежность архитектуры приложения. Полученные навыки работы с событийной моделью Python, глубоким конфигурированием виджетов и управлением состояниями объектов составляют прочный фундамент для разработки более сложных систем обработки данных и проектирования пользовательских интерфейсов

Список использованной литературы

1. Лутц М. Программирование на Python том II / М. Лутц – 4-е издание. – Пер. с англ. – СПб.: Символ-Плюс, 2011. – 992 с., ил. ISBN 978-5-93286-211-7
2. Мартин Р. Чистый код: создание, анализ и рефакторинг. Библиотека программиста. / Р. Мартин – СПб.: Питер, 2013. – 464 с.: ил. ISBN 978-5-496-00487-9
3. PEP 8 : Style Guide for Python Code : Python Enhancement Proposal // Python.org. – URL: <https://peps.python.org/pep-0008/> (дата обращения: 10.04.2026). – Яз. англ. – Режим доступа: свободный. – Текст : электронный.
4. Shipman, J. W. Tkinter 8.5 reference: a GUI for Python / J. W. Shipman. – New Mexico: New Mexico Tech Computer Center, 2013. – URL: <https://anzeljg.github.io/rin2/book2/2405/docs/tkinter/index.html> (дата обращения: 08.04.2026). – Яз. англ. – Режим доступа: свободный. – Текст : электронный.
5. Tkinter: Python interface to Tcl/Tk // docs.python.org. – URL: <https://docs.python.org/3/library/tkinter.html> (дата обращения: 10.04.2026). – Яз. англ. – Режим доступа: свободный. – Текст : электронный.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ 1. Исходный код программного модуля «PoLLelya»

```
import tkinter as tk
from tkinter import filedialog, messagebox, ttk
from tkinter.scrolledtext import ScrolledText

class PoLLelya(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("PoLLelya")
        self.geometry("1200x900")
        self.is_dark_theme = True
        self.filename = None
        self.last_saved_content = ""
        # current_tags
хранит активные стили для БУДУЩЕГО ввода
        self.current_tags = set()
        self.set_theme_colors()
        self.setup_ui()
        self.setup_bindings()
        self.protocol("WM_DELETE_WINDOW",
self.on_closing)

    def set_theme_colors(self):
        if self.is_dark_theme:
            self.colors = {"bg": "#1a1a1e", "surface":
"#252529", "accent": "#7c4dff", "text": "#e1e1e1",
"gray": "#626266",
"border": "#333338"}
        else:
            self.colors = {"bg": "#f0f0f0", "surface":
"#ffffff", "accent": "#7c4dff", "text": "#1a1a1e",
"gray": "#a0a0a0",
"border": "#cccccc"}

    def setup_ui(self):
        # 1. Запоминаем данные,
если редактор уже существовал
        saved_data = None
        current_font = "Times New Roman"
        current_size = "14"

        if hasattr(self, 'editor'):
```

```

        saved_data = self.editor.dump("1.0", "end-
1c", tag=True, text=True)
        current_font = self.font_combo.get()
        current_size = self.size_combo.get()
        for widget in self.wininfo_children():
            widget.destroy()

        self.configure(bg=self.colors["bg"])

        # Сайдбар
        self.sidebar = tk.Frame(self,
bg=self.colors["surface"], width=70)
        self.sidebar.pack(side=tk.LEFT, fill=tk.Y)

        for icon, cmd in [("\u270f", self.new_file),
("\u2705", self.save_file), ("\u2704", self.open_file), ("\u25cf",
self.toggle_theme)]:
            btn = tk.Label(self.sidebar, text=icon,
bg=self.colors["surface"], fg=self.colors["gray"],
font=("Segoe UI Symbol",
20), pady=15, cursor="hand2")
            btn.pack(fill=tk.X)
            btn.bind("<Button-1>", lambda e, c=cmd:
c())

        right_side = tk.Frame(self,
bg=self.colors["bg"])
        right_side.pack(side=tk.LEFT, fill=tk.BOTH,
expand=True)

        # Тулбар
        toolbar_container = tk.Frame(right_side,
bg=self.colors["bg"], pady=20, padx=40)
        toolbar_container.pack(fill=tk.X)
        self.toolbar = tk.Frame(toolbar_container,
bg=self.colors["surface"], padx=15, pady=8)
        self.toolbar.pack(fill=tk.X)

        self.font_combo = ttk.Combobox(self.toolbar,
values=["Times New Roman", "Arial", "Courier New",
"Consolas"],
width=15,
state="readonly")
        self.font_combo.set(current_font)
        self.font_combo.pack(side=tk.LEFT, padx=10)

```

```

        self.font_combo.bind("<<ComboboxSelected>>",
lambda e: self.on_style_change())

        self.size_combo = ttk.Combobox(self.toolbar,
values=[8, 9, 10, 11, 12, 14, 16, 18, 20, 24, 30, 36,
48, 60, 72],
width=5)
        self.size_combo.set(current_size)
        self.size_combo.pack(side=tk.LEFT, padx=10)
        self.size_combo.bind("<<ComboboxSelected>>",
lambda e: self.on_style_change())

        self.btns = {}
        for label, tag in [("B", "bold"), ("I",
"italic"), ("U", "underline")]:
            btn = tk.Button(self.toolbar, text=label,
relief="flat", bd=0, font=("Segoe UI", 11), width=3,
command=lambda t=tag:
self.toggle_tag(t))
            btn.pack(side=tk.LEFT, padx=2)
            self.btns[tag] = btn

        # Цвета
        colors_preset = ["#e1e1e1" if
self.is_dark_theme else "#1a1a1a", "#ff4f4f",
"#4fff7c", "#4f7cff", "#ffcf4f"]
        for c in colors_preset:
            btn = tk.Frame(self.toolbar, bg=c,
width=20, height=20, cursor="hand2",
highlightthickness=1,
highlightbackground=self.colors["border"])
            btn.pack(side=tk.LEFT, padx=3)
            btn.bind("<Button-1>", lambda e, color=c:
self.change_color(color))

        # Редактор
        self.editor = ScrolledText(right_side,
bg=self.colors["surface"], fg=self.colors["text"],
font=(current_font,
int(current_size)),
bd=0, padx=40,
pady=40, insertbackground=self.colors["accent"],
undo=True)
        self.editor.pack(fill=tk.BOTH, expand=True,

```

```
padx=40, pady=(0, 40))
```

```
# 2.
```

```
Восстанавливаем контент и визуальное состояние кнопок
```

```
if saved_data:
```

```
    self.restore_content(saved_data)
```

```
self.update_btn_ui()
```

```
self.editor.focus_set()
```

```
def restore_content(self, data):
```

```
    self.editor.delete("1.0", tk.END)
```

```
    # Сначала настраиваем стандартные теги
```

```
    self.editor.tag_configure("underline",
```

```
underline=True)
```

```
    for type, value, index in data:
```

```
        if type == "text":
```

```
            self.editor.insert(index, value)
```

```
        elif type == "tagon" and value not in  
["sel", "insert"]:
```

```
            #
```

```
Восстанавливаем конфигурацию тега на лету
```

```
        if value.startswith("f_"):
```

```
            parts = value.split("_")
```

```
            if len(parts) >= 5:
```

```
                f_name = parts[1].replace("-",  
" ")
```

```
                f_weight, f_slant, f_size =  
parts[2], parts[3], parts[4]
```

```
                self.editor.tag_configure(value,  
e, font=(f_name, int(f_size), f_weight, f_slant))
```

```
            elif value.startswith("fg_"):
```

```
                color = value.split("_")[1]
```

```
                self.editor.tag_configure(value,  
foreground=color)
```

```
        self.editor.tag_add(value, index)
```

```
def get_current_settings(self):
```

```
    f_name = self.font_combo.get()
```

```
    f_size = self.size_combo.get() if
```

```
self.size_combo.get().isdigit() else "14"
```

```
    weight = "bold" if "bold" in self.current_tags  
else "normal"
```

```

        slant = "italic" if "italic" in
self.current_tags else "roman"
        return f_name, f_size, weight, slant

    def on_style_change(self):
        f_name, f_size, weight, slant =
self.get_current_settings()
        try:
            s, e = self.editor.index("sel.first"),
self.editor.index("sel.last")
            self.apply_font_style(s, e, f_name,
f_size, weight, slant)
        except tk.TclError:
            pass
        self.editor.focus_set()

    def toggle_tag(self, tag):
        try:
            s, e = self.editor.index("sel.first"),
self.editor.index("sel.last")
            if tag == "underline":
                if tag in self.editor.tag_names(s):
                    self.editor.tag_remove(tag, s, e)
                else:
                    self.editor.tag_add(tag, s, e)
            else:
                tags = self.editor.tag_names(s)
                is_b = any("bold" in t for t in tags
if t.startswith("f_"))
                is_i = any("italic" in t for t in tags
if t.startswith("f_"))

                new_b = (not is_b) if tag == "bold"
else is_b
                new_i = (not is_i) if tag == "italic"
else is_i

                f_name, f_size, _, _ =
self.get_current_settings()
                self.apply_font_style(s, e, f_name,
f_size, "bold" if new_b else "normal",
                                "italic" if
new_i else "roman")
        except tk.TclError:
            if tag in self.current_tags:

```



```

        self.current_tags.remove(tag)
    else:
        self.current_tags.add(tag)
        self.update_btn_ui()
self.editor.focus_set()

def update_btn_ui(self):
    for tag, btn in self.btns.items():
        active = tag in self.current_tags
        btn.config(
            bg=self.colors["accent"] if active
else self.colors["surface"],
            fg="white" if active else
self.colors["text"]
        )

    def apply_font_style(self, start, end, name, size,
weight, slant):
        tag_name = f"f_{name.replace('&apos; &apos;;',
&apos;-&apos;)}_{weight}_{slant}_{size}"
        self.editor.tag_configure(tag_name,
font=(name, int(size), weight, slant))
        for t in self.editor.tag_names(start):
            if t.startswith("f_"):
                self.editor.tag_remove(t, start, end)
        self.editor.tag_add(tag_name, start, end)

    def change_color(self, color):
        tag_name = f"fg_{color}"
        self.editor.tag_configure(tag_name,
foreground=color)
        try:
            s, e = self.editor.index("sel.first"),
self.editor.index("sel.last")
            for t in self.editor.tag_names(s):
                if t.startswith("fg_"):
self.editor.tag_remove(t, s, e)
            self.editor.tag_add(tag_name, s, e)
        except tk.TclError:
            self.current_tags = {t for t in
self.current_tags if not t.startswith("fg_")}
            self.current_tags.add(tag_name)
            self.editor.focus_set()

    def handle_keypress(self, event):

```

```

        if event.char and ord(event.char) > 31:
            self.editor.insert(tk.INSERT, event.char)
            idx = self.editor.index("insert-1c")

            f_name, f_size, weight, slant =
self.get_current_settings()
            tag_name = f"f_{f_name.replace('&apos;','&apos;-&apos;')}_{'weight'}_{'slant'}_{'f_size'}"
            self.editor.tag_configure(tag_name,
font=(f_name, int(f_size), weight, slant))
            self.editor.tag_add(tag_name, idx)

            if "underline" in self.current_tags:
                self.editor.tag_add("underline", idx)
            for t in self.current_tags:
                if t.startswith("fg_"):
                    self.editor.tag_add(t, idx)
            return "break"

    def setup_bindings(self):
        self.bind("<Control-b>", lambda e:
self.toggle_tag("bold"))
        self.bind("<Control-i>", lambda e:
self.toggle_tag("italic"))
        self.bind("<Control-u>", lambda e:
self.toggle_tag("underline"))
        self.editor.bind("<KeyPress>",
self.handle_keypress)

    def toggle_theme(self):
        self.is_dark_theme = not self.is_dark_theme
        self.set_theme_colors()
        self.setup_ui()
        self.setup_bindings()

    def is_modified(self):
        return self.editor.get("1.0",
tk.END).strip() != self.last_saved_content.strip()

    def new_file(self):
        if self.is_modified() and
messagebox.askyesno("Сохранение",
"Сохранить изменения?"):
            self.save_file()
            self.editor.delete("1.0", tk.END)

```

```

        self.last_saved_content = ""
        self.filename = None

    def save_file(self):
        if not self.filename:
            self.filename =
filedialog.asksaveasfilename(defaultextension=".txt")
        if self.filename:
            with open(self.filename, "w",
encoding="utf-8") as f:
                c = self.editor.get("1.0", tk.END)
                f.write(c)
                self.last_saved_content = c.strip()

    def on_closing(self):
        if self.is_modified():
            ans = messagebox.askyesnocancel("Выход",
"Сохранить изменения?")
            if ans is True:
                self.save_file()
                self.destroy()
            elif ans is False:
                self.destroy()
        else:
            self.destroy()

    def open_file(self):
        p = filedialog.askopenfilename()
        if p:
            with open(p, "r", encoding="utf-8") as f:
                c = f.read()
                self.editor.delete("1.0", tk.END)
                self.editor.insert("1.0", c)
                self.filename = p
                self.last_saved_content = c.strip()

if __name__ == "__main__":
    PoLLelya().mainloop()

```

ПРИЛОЖЕНИЕ 2. Демонстрация работы интерфейса и функций редактора.



